quickstart

May 8, 2026

This notebook shows minimal end-to-end examples for setting up an AI client in `aidu`. The client is typically a thin abstraction layer over an underlying `AI API` and can target a local or remote AI service.

1 Use a classical local AI service

To avoid requiring an OpenAI API key, the first example uses `SymPyClient`. This is a classical AI workflow, so no LLM API access is needed yet. Everything runs on your local machine.

How to work with Jupyter

1. Run the cells from top to bottom.
2. Update the `problem` string in Step 4 if you want to try another expression.
3. Inspect the parsed JSON output in Step 5.

The default example computes:

- Derivative of `7x^2 + 3x - 5` with respect to `x`.

1.1 Step 1: Import global packages

We import `json` to parse structured content returned by the agent. `time` helps us to measure how long things take. The `rich` package is used to print results in a pretty readable format. `IPython.display` contains display helpers.

[1]:
```
import json
import time

from rich.console import Console
from IPython.display import Math, Latex
```

1.2 Step 2: Import local packages

Each interaction carries a structured `Context`, which includes a `Trace` of messages.

The local client is defined in `clients.sympy`.

[2]:
```
from aidu.ai.llm.client import Context
from aidu.ai.llm.client import Trace, State, Control
import aidu.ai.llm.clients.sympy as sp
```

1.3 Step 3: Create the SymPy client

We create a local math client that returns JSON output, then initialize a rich console for readable printing.

[3]:
```
console = Console()
client = sp.SymPyClient(
    "sympy",
    config={"enforce_json": True},
    process=sp.solve_math_problem_with_sympy,
)
```

1.4 Step 4: Define the math problem

Set the expression you want to solve and build the user message payload.

[4]:
```
# Step 4: Define the math problem and create the message
problem = "diff(7x^2 + 3x - 5, x)"
message = {"role": "user", "content": problem}
```

1.5 Step 5: Run the request and inspect the result

Send the message with a fresh context, parse the JSON response, and print it.

[5]:
```
response = client.chat(message, Context())
content = json.loads(response["content"])

console.print(content, width=80)
```

```
{
    'type': 'derivative',
    'expression': '7*x**2 + 3*x - 5',
    'result': '14*x + 3',
    'latex': '14x+3',
    'message': 'When we differentiate $7x^{2}+3x-5$, we get $14x+3$ from SymPy.'
}
```

2 Use an LLM service

We can solve the same problem with a language model.

API key required

Define your key in `.env` before running any cells:

```
OPENAI_API_KEY=your_api_key_here
```

2.1 Step 1: Load API key and import the LLM client

We load environment variables from `.env`, then fetch `OPENAI_API_KEY` for the OpenAI-backed client.

[6]:
```
import os
from dotenv import load_dotenv

import aidu.ai.llm.clients.openai as openai_llm

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")
assert api_key, "Missing OPENAI_API_KEY in .env"
```

2.2 Step 2: Create the LLM client and run the same task

This mirrors the classical example above. We create an LLM client and request the answer from the OpenAI service.

[7]:
```
gpt_4o_mini_client = openai_llm.OpenAIClient(
    "gpt-4o-mini",
    config={"enforce_json": True},
    api_key=api_key,
)
```

2.3 Step3: Create the prompt

In the classic example the result had been defined via python function, where now we have to define the task in plain english.

[8]:
```
prompt = (
        "You are a deterministic math solver like a sympy solver. "
        "Solve the given math problem and return strict JSON with exactly these␣
 ↪keys: "
        "type, expression, result, latex, message. "
        "Do not add extra keys or markdown."
    )
```

2.4 Step 4: Create the Context

With that prompt in place, we can define the first message, which will be part of our context which is threated through the dialog- We use the same user `message` as above. The LLM responses with a `dict`, which gives us is actual answer under `content` and some helpful side informations, like tokens used. The LLM is following our instruction and encodes its output in JSON.

[9]:
```
system_message = {
    "role": "system",
    "content": prompt,
}
```

```
context = Context(trace=Trace(messages=[system_message]))

llm_response = gpt_4o_mini_client.chat(message, context)

console.print(llm_response)
```

```
{
    'content': '{\n  "type": "derivative",\n  "expression": "7x^2 + 3x - 5",\n␣
 ↪"result": "14x + 3",\n  "latex":
"14x + 3",\n  "message": "The derivative of the expression with respect to x has␣
 ↪been calculated."\n}',
    'role': 'assistant',
    'prompt_tokens': 71,
    'completion_tokens': 67,
    'total_tokens': 138,
    'cost_usd': 5.0849999999999996e-05,
    'model': 'gpt-4o-mini'
}
```

Finally, we parse the JSON response and pretty print the json result.

[10]:
```
llm_content = json.loads(llm_response["content"])

console.print(llm_content)
```

```
{
    'type': 'derivative',
    'expression': '7x^2 + 3x - 5',
    'result': '14x + 3',
    'latex': '14x + 3',
    'message': 'The derivative of the expression with respect to x has been␣
 ↪calculated.'
}
```

3 Finding Zeros

Now lets see where are the limits of the LLM. First we build a test equation.

3.1 Using SymPy

[11]:
```
from sympy import symbols, expand, sympify, latex

x = symbols('x')

equation = "(sqrt(3)-x)*(4+x)*(3.5-x)"
```

```
expr = sympify(equation)

display(Math(r"\text{factored: }\quad " + latex(expr)))

result = expand(expr)

display(Math(r"\text{expanded: }\quad " + latex(result)))
```

factored: $(3.5 - x)\left(-x + \sqrt{3}\right)(x + 4)$

expanded: $x^3 - \sqrt{3}x^2 + 0.5x^2 - 14.0x - 0.5\sqrt{3}x + 14.0\sqrt{3}$

[12]:
```
message = {"role": "user", "content": f"solve({result}, x)"}

# timed llm execution
start = time.perf_counter()
response = client.chat(message, Context())
end = time.perf_counter()

content = json.loads(response["content"])

console.print(f"Sympy call took {end - start:.3f} seconds")
display(Math(r"\text{Zeros: }\quad " +content["latex"]))
```

```
Sympy call took 0.086 seconds
```

Zeros: $-4, \frac{7}{2}, \sqrt{3}$

3.2 Using GPT 4o mini

GPT 4a mini is a good compromise between speed and precision, but sadly not in math.

[13]:
```
system_message = {
    "role": "system",
    "content": prompt,
}

context = Context(trace=Trace(messages=[system_message]))

# timed llm execution
start = time.perf_counter()
llm_response = gpt_4o_mini_client.chat(message, context)
end = time.perf_counter()

llm_content = json.loads(llm_response["content"])
```

```
console.print(f"Used tokens: {llm_response['total_tokens']} for␣
 ↪{float(llm_response['cost_usd'])*1000.:.2f} ¢")


console.print(f"LLM call took {end - start:.3f} seconds")

display(Math(r"\text{Zeros: }\quad " + llm_content["latex"]))
```

```
Used tokens: 293 for 0.13 ¢


LLM call took 5.194 seconds
```

Zeros: $x^3 - \sqrt{3}x^2 + 0.5x^2 - 14.0x - 0.5\sqrt{3}x + 14.0\sqrt{3} = 0 \Rightarrow x = 7.0, -1.0, 2.0$

3.3 Using GPT 5 mini

Now we use the flagship model `gpt-5`. It can do proper math, even its mini variant, but it needs **many tokens**, it is **very slow** (800 times slower than sympy locally) and **costly** (40 times the cost of GPT 4o mini), and we have to assume the model is offloading math to cumputer algebra helpers, but it has to think a lot.

[14]:
```
gpt_5_mini_client = openai_llm.OpenAIClient(
    "gpt-5-mini",
    config={"enforce_json": True},
    api_key=api_key,
)
if False: # Set to True once
    # timed llm execution
    start = time.perf_counter()
    llm_response = gpt_5_mini_client.chat(message, context)
    end = time.perf_counter()

    console.print(f"Used tokens: {llm_response['total_tokens']} for␣
 ↪{float(llm_response['cost_usd'])*1000.:.2f} ¢")

    llm_content = json.loads(llm_response["content"])

    console.print(f"LLM call took {end - start:.3f} seconds")
    display(Math(r"\text{expanded: }\quad " + llm_content["latex"]))
else:
    console.print(f"  You need to allow this code explicitly as it is expensive.
 ↪"
                  f"You will see the right solution with Tokens: 2076, COsts 4␣
 ↪¢, Time: 17 sec.")
```

```
 You need to allow this code explicitly as it is expensive.You will see the␣
 ↪right solution with Tokens: 2076,
COsts 4 ¢, Time: 17 sec.
```

3.4 Using a Google model

We can also try a Google model, which is 2x faster than GPT 5 mini, but still much slower than SymPy. It typically does not even try to solve the problem, but just outputs the original equation somewhat reformated in LaTeX format, which is not what we wanted.

[15]:
```
import aidu.ai.llm.clients.google as google_llm

google_api_key = os.getenv("GOOGLE_API_KEY")
gemini_2_5_flash_lite = google_llm.GoogleClient(
    "gemini-2.5-flash-lite",
    config={"enforce_json": True},
    api_key=google_api_key,
)

# console.print(message)
# console.print(context)

# timed llm execution
start = time.perf_counter()
llm_response = gemini_2_5_flash_lite.chat(message, context)
end = time.perf_counter()

# console.print(llm_response)
```

[16]:
```
llm_content = json.loads(llm_response["content"])

console.print(f"Used tokens: {llm_response['total_tokens']} for␣
 ↪{float(llm_response['cost_usd'])*1000.:.2f} ¢")


console.print(f"LLM call took {end - start:.3f} seconds")

display(Math(r"\text{Zeros: }\quad " + llm_content["latex"]))
```

```
Used tokens: 285 for 0.08 ¢


LLM call took 1.489 seconds
```

Zeros: $\quad x^3 + \left(0.5 - \sqrt{3}\right) x^2 - \left(14 + \frac{\sqrt{3}}{2}\right) x + 14\sqrt{3}$